

VINUNIVERSITY

Model Serving & Inference Optimization

AICB-P2T2 · Ngày 20 · Chương 4: Hạ Tầng

Giảng viên

VinUniversity · Phase 2 · Track 2 · Tuần 4

HÃY SUY NGHĨ...

“Model accuracy 95% nhưng latency 3 giây. User đợi không nổi, churn tăng 40%. Model tốt nhưng serve chậm = product thất bại.”

Giữ câu hỏi này trong đầu khi học bài hôm nay

Nội Dung Bài Học

1. Bối Cảnh & Vocabulary (latency, pre-LLM era)
2. Quantization: FP16/FP8/AWQ/GGUF/NVFP4
3. KV Cache & Attention Optimization
4. Single-Node Serving Stack 2026 (8 engines)
5. Distributed & Multi-Tenant Serving
6. Auto-scaling & Operations
7. Edge & Hardware Landscape
8. Production SLA (Goodput@SLO)
9. Lab 20 + Milestone 1

Mục tiêu bài học

Sau buổi học này, bạn sẽ:

1. Phân biệt **Throughput vs Goodput@SLO**, đọc TTFT/TPOT trên dashboard production
2. Áp dụng quantization (FP16/FP8/AWQ 4-bit/NVFP4/GGUF) để giảm memory & tăng throughput
3. Hiểu KV Cache, PagedAttention, RadixAttention, FlashAttention 3/4, MHA→MLA
4. So sánh 8 serving engines (vLLM, SGLang, NVIDIA Dynamo, llm-d, LMDeploy, TensorRT-LLM, Ollama, llama.cpp)
5. Chọn đúng parallelism strategy (TP/PP/EP/DP) cho workload distributed
6. Hoàn thành Lab 20 (llama.cpp tuning bonus) và submit Milestone 1

> Foundations → Quantization → KV/Attention → Single-Node Serving → Distributed → Auto-scale → Edge → SLA → Lab 20 + M1

Deliverable Cuối Ngày

Optimized inference stack + Lab 20 report + Milestone 1 demo

- Benchmark report: FP16 vs FP8 vs AWQ 4-bit vs continuous batching
- Load test: 100 concurrent users, P50/P95/P99 latency
- Lab 20 report (`benchmarks/results.md` với P50/P95/P99 + bonus tuning notes)
- Milestone 1: AI infrastructure platform demo (N16–N19)

Latency Taxonomy: TTFT · TPOT · Goodput

- **TTFT** *(Time To First Token)*: từ request đến token đầu tiên — phụ thuộc prefill compute + queue wait
- **TPOT** *(Time Per Output Token)* = ITL: khoảng cách đều giữa mỗi output token
- **E2E Latency** = TTFT + TPOT $\times$ (N – 1); SLO thường ở P95/P99

- **Throughput**: tokens/s toàn hệ thống ở saturation, không có SLO constraint
- **Goodput**: req/s *thỏa mãn* TTFT+TPOT SLO — **metric production quan trọng nhất**
- **Queue Depth**: requests đang chờ prefill — chỉ báo saturation

Ví dụ thực tế — H100 · Llama-3-70B · batch 32: **TTFT** $\approx$ 450 ms · **TPOT** $\approx$ 25 ms · Throughput 1,800 tok/s · Goodput @SLO(TTFT $<$ 1s, TPOT $<$ 50ms) $\approx$ 1,200 tok/s

Lưu ý: **Throughput @saturation** $\neq$ **Goodput @SLO.** Báo cáo chỉ throughput mà bỏ qua SLO constraint là misleading — luôn report goodput cho production.

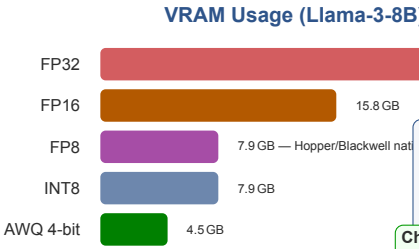
Pre-LLM Era: Serving Infrastructure 2017–2022

- **TF Serving** (Google 2017): `SavedModel`, gRPC, versioned endpoints, static batch
- **Triton Inference Server** (NVIDIA 2019): multi-framework, dynamic batching per model
- **ONNX Runtime** (2019): cross-framework export, CPU/GPU/NPU optimized kernels
- **TorchServe** (Facebook 2020): MAR archives, REST API, multi-model serving
- **BentoML** (2020): Python-native packaging, pluggable runtimes

- **Memory fragmentation**: KV cache cần contiguous block cố định — 60–80% VRAM waste
- **Static batching**: chờ đủ batch, thêm 200–500 ms latency
- **No token streaming**: client chờ toàn bộ response trước khi nhận
- **No continuous batching**: 1 long request block toàn queue
- **Fixed-length I/O**: không xử lý được variable-length generation

The Shift (Jun 2023) — vLLM **PagedAttention**: KV cache qua virtual memory pages (non-contiguous, no fragmentation) + **continuous batching** (requests vào/ra liên tục) $\rightarrow$ LLM serving era. 24$\times$ throughput vs naive HF Transformers.

Quantization: Precision vs Performance



FP8: $<$1% drop, 2$\times$ memory vs FP16 (Hopper+ native)
AWQ 4-bit: $\sim$1pt MMLU drop trên 8B+, lớn hơn với $<$7B
NVFP4 (Blackwell): 3.5$\times$ vs

Chọn quantization:
Production Hopper: FP8 / AWQ 4-bit
Production Blackwell: NVFP4 (default)
Max quality: BF16 / FP16
Edge/laptop: GGUF Q4_K_M (i-quants nếu $<$Q4)

Quantization for Memory: GGUF · NF4 · GPTQ · AWQ

VINUNIVERSITY

Format	BPW	8B VRAM	70B VRAM	Quality
FP16	16	15.8 GB	140 GB	Baseline
FP8	8	7.9 GB	70 GB	$<$1% drop
AWQ 4-bit	4.5	4.5 GB	40 GB	$\sim$1pt MMLU
GPTQ 4-bit	4.0	4.0 GB	35 GB	$\sim$1pt MMLU
GGUF Q4_K_M	4.5	4.8 GB	43 GB	$\sim$1pt MMLU
GGUF Q2_K	2.6	2.7 GB	24 GB	Noticeable
NF4 (bnb)	4.0	4.0 GB	36 GB	$\sim$1pt MMLU

- **GPTQ**: inverse Hessian layer-by-layer (128 calib. samples). Chậm quantize, nhanh inference.
- **AWQ**: tìm salient weights (high activation magnitude), scale trước INT4 rounding. Acc $>$ GPTQ cùng bits.
- **NF4** (bitsandbytes): 4-bit NormalFloat, optimal cho normal-distributed weights. Double quant: constants FP32$\rightarrow$FP8.
- **GGUF k-quant**: Q4_K_M = mixed Q4/Q6 per tensor (attn vs FFN). `k`=better quant, `m`=medium size.

- GPU cloud (Hopper): **FP8** — best quality/perf
- GPU VRAM tight: **AWQ 4-bit** — tốt nhất ở 4-bit
- CPU-only inference: **GGUF Q4_K_M** — recommended
- Cực kỳ constrained: **GGUF Q2_K** — last resort
- Fine-tune trên 1 GPU: **NF4 + QLoRA**

Lưu ý: $<$7B models mất quality nhanh hơn

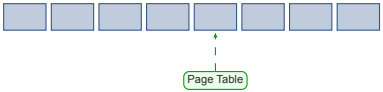
KV Cache & PagedAttention

Traditional: Contiguous



Wasted!

PagedAttention: Paged

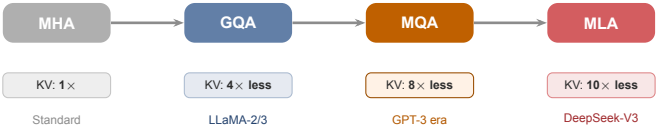


No waste

- KV cache như virtual memory pages
- 24× vs naive HF Transformers
- Dynamic memory allocation

- Tái sử dụng KV qua radix tree (prefix sharing)
- Lý tưởng cho RAG, multi-turn, agents
- Engineering chi tiết: xem §3 Prefix Caching

Attention Architecture: MHA → MLA & Long-Context



- Compress Q/K/V xuống latent vector nhỏ trước attention
- DeepSeek-V3: 10× ít KV memory vs standard MHA
- Kernel: FlashMLA, CutlassMLA, FlashInfer (2025)
- Cho phép context dài hơn trên cùng VRAM

- **YaRN**: RoPE interpolation, không cần retrain
- **StreamingLLM**: attention sinks, ∞ context
- **Jamba** (AI21 Labs): SSM/Transformer hybrid, 256K ctx
- FA3 + MLA backends → kernel cho context dài (xem §3)

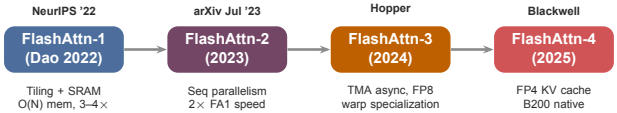
Speculative Decoding & Continuous Batching

- Draft model sinh 4–8 tokens, target verify song song trên cùng 1 forward pass
- **EAGLE-3** (NeurIPS '25): 3.0–6.5×, +20–40% so EAGLE-2
- **DeepSeek MTP** (Multi-Token Prediction): ∼1.8×, acceptance rate >80%
- **Lookahead Decoding**: self-drafting khi không có draft model
- Tích hợp sẵn trong vLLM, SGLang, TensorRT-LLM

- **Static batching** (legacy): chờ đủ batch → +200–500 ms padding
- **Continuous batching**: requests vào/ra mỗi step, no padding → **5**× latency giảm
- **Token streaming**: client nhận từng token — TTFT cảm giác ↓
- vLLM (continuous batching), TensorRT-LLM (in-flight batching) — thuật ngữ tương đương
- SGLang *piecewise CUDA graph* cho variable-length batch

Spec-Decode CLI (SGLang) — `--speculative-algorithm EAGLE3 --speculative-num-steps 5 --speculative-eagle-topk 4`. Yêu cầu draft model checkpoint hoặc model có MTP head.

FlashAttention: IO-Aware Attention (FA1→FA4)



- Standard attn: Q×K ($N^2 \times d$) viết HBM → đọc lại softmax → đọc lại ×V — 3 HBM round trips
- FA: tile Q/K/V vào SRAM, online softmax, ghi output **1 lần duy nhất**
- Memory: O(N) thay vì $O(N^2)$ — context dài không OOM
- FA3: Hopper TMA async pipeline + FP8. FA4: Blackwell FP4 KV, SGLang auto-selects

- `torch.nn.attention.flex_attention`: BlockMask API
- Express: causal, sliding window, document boundary, prefix+causal
- Compile qua `torch.compile` → Triton kernel, không cần CUDA custom
- Tích hợp: PyTorch `F.scaled_dot_product_attention`, vLLM, SGLang custom patterns

Model Compilation: torch.compile + CUDA Graphs + TensorRT

- Capture computation graph qua `torch.fx`; TorchInductor tạo Triton kernels
- Kernel fusion: nhiều element-wise ops $\rightarrow$ 1 kernel, ít HBM reads hơn
- `mode="max-autotune"`: tìm optimal kernel config (compile chậm, run nhanh)
- `mode="reduce-overhead"`: loại Python overhead nhanh
- `dynamic=True`: variable shapes không trigger recompile
- Speedup: 1.1–1.5$\times$ trên LLM decode phase

- **Record**: chạy 1 forward pass, capture GPU command stream
- **Replay**: skip Python overhead mọi lần sau (0.5–2 ms/step tiết kiệm)
- LLM decode = cùng ops lặp N_tokens lần $\rightarrow$ CUDA graph lý tưởng
- **vLLM v1**: decode dùng CUDA graph replay; prefill chạy eager (variable shape)
- **SGLang**: "piecewise CUDA graph" cho mixed static/dynamic batch sizes
- Cộng thêm 10–20% throughput trên mọi optimization khác

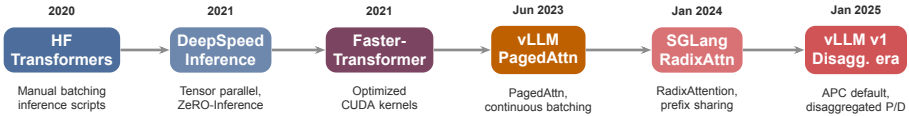
TensorRT Compilation — ONNX $\rightarrow$ layer fusion $\rightarrow$ kernel selection $\rightarrow$ FP8/INT8 calibration $\rightarrow$ `.trt` engine. 3–5$\times$ vs vanilla PyTorch. Compile time: 5–30 min/model. Dùng trong TensorRT-LLM và NVIDIA Triton Inference Server.

Serving Stack 2026: 8 Engines That Matter

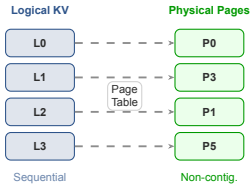
Engine	Ưu điểm chính	Best for	API
vLLM	PagedAttention, APC + chunked prefill (v1 default)	LLM production	OpenAI-compat
SGLang	RadixAttention, structured gen, MLA backend	Multi-turn / chat	OpenAI-compat
NVIDIA Dynamo	Disaggregated P/D, NIM-ready	Multi-tenant cloud	OpenAI-compat
llm-d	K8s-native, KV-aware routing	Production at scale	OpenAI-compat
LMDeploy	TurboMind engine, hiệu suất cao	High throughput	OpenAI-compat
TensorRT-LLM	NVIDIA native, FP8/FP4 optimized	NVIDIA GPU fleet	Triton backend
Ollama	1 lệnh: `ollama run` (wraps llama.cpp)	Local dev/testing	REST
llama.cpp	GGUF native, CPU+GPU mixed offload, Apple Metal	Local / CPU / edge	REST / OpenAI-compat

Lưu ý: Production 2026: vLLM v1 hoặc SGLang. Disaggregated scale: llm-d / Dynamo. Local CPU/Mac: llama.cpp. Container: Ollama. NVIDIA fleet: TensorRT-LLM.

Inference Engine Evolution: 2020 → 2025



2020–2022: manual/static batching, CUDA kernels — framework-level optimizations. **Jun 2023:** PagedAttention → continuous batching → 24× throughput jump, ecosystem convergence. **2024–25:** prefix sharing + disaggregated P/D → TTFT và goodput là SLO first-class.



- PagedAttention: KV như virtual memory pages, **24**× vs HF naive
- Block size: 16 tokens/page (default). FCFS scheduler + preemption
- Continuous batching: requests join/leave mid-generation
- Prefix caching opt-in (`--enable-prefix-caching`)

- Unified memory pool: KV cache + activations trong cùng pool
- **APC ON by default**: Automatic Prefix Caching, không cần flag
- Chunked prefill default: chia prefill thành chunks, interleave với decode
- Prefix-aware scheduler. **1.7**× v0 throughput.

Prefix Caching: RadixAttention, APC, HiCache & Pricing

- **vLLM v1 APC**: Automatic Prefix Caching, ON mặc định
- **RadixAttention** (SGLang): radix-tree (prefix trie), cache hit = skip prefill toàn bộ shared prefix
- **LMCache**: cross-instance KV sharing (CPU/disk)
- **Mooncake KVCache**: global pool trên disaggregated cluster
- Tiết kiệm prefill: -70% TTFT trên repeated system prompts (RAG, agents, multi-turn)

- **Anthropic**: cached input $-$**90%** (Claude 3.5/3.7)
- **DeepSeek**: cache-hit **10**$\times$ rẻ hơn ($0.014 vs $0.14/M tok)
- **OpenAI**: cached input $\approx$50% off (GPT-4o)
- **Google**: context caching **25**$\times$ cheaper/token (Gemini 1.5/2.0)

Tier 1 GPU VRAM (active, hot) $\rightarrow$ **Tier 2** Host RAM (spillover) $\rightarrow$ **Tier 3** External storage (HF3FS, Mooncake, disk). Attach/detach backend không cần restart. Long-context: vượt GPU VRAM limit. Multi-turn: reuse KV qua nhiều turns.

Attention Backend Selection: FA3 / FA4 / FlashInfer / FlashMLA

- H100/H200 (Hopper, CUDA 12.3+) → **`fa3`**
- B200 (Blackwell) → **`trtllm_mha`** hoặc **`fa4`**
- A100/A40 → **`flashinfer`**
- DeepSeek V3/R1 MLA → **`flashmla`** (page=64)
- ROCm / Ascend / CPU → **`triton`** (cross-platform fallback)

Override: `--attention-backend {fa3|fa4|flashinfer|trtllm_mha|flashmla|triton}`

- **FA3**: TMA async, FP8 KV, warp specialization — Hopper native
- **FA4**: FP4 KV cache — Blackwell SM100 (2025)
- **FlashInfer**: page-size >1, FP8 KV, spec-decode topk >1, sliding window
- **TRTLLM-MLA**: DeepSeek MLA optimized, verified spec-decode
- **Triton**: cross-platform fallback (ROCm, Ascend, NPU, CPU)

MLA Backends (DeepSeek V3/R1) — FlashMLA · CutlassMLA · FlashInfer-MLA · TRTLLM-MLA — **3.1**× throughput vs MHA, **10**× less KV memory. Tự động chọn theo GPU + model; override chỉ khi benchmark/debug.

Structured Generation: XGrammar, Tool & Reasoning Parsers

- Formats: JSON Schema, EBNF, Regex, Pydantic model
- Grammar backends: **XGrammar** (default, fastest), Outlines, Llguidance
- Engine support: SGLang `--grammar-backend xgrammar`, vLLM `--guided-decoding-backend xgrammar`
- API: OpenAI-compat `response_format={json_schema}`
- Reasoning models: constraint áp dụng *sau* `<think>...</think>`

- **Tool Parser** `--tool-call-parser [model]`: 15+ models (DeepSeek, Llama-3.1/4, Qwen, Mistral, Kimi-K2); streaming args incrementally
- **Reasoning Parser** `--reasoning-parser [model]` (deepseek-r1, qwen3, kimi_k2): trích `<think>` → `reasoning_content` + `content` tách biệt
- Kết hợp: `--reasoning-parser deepseek-r1 --tool-call-parser kimi_k2`

Workflow — Prompt → `<think>` (free reasoning) → grammar-constrained output → response: `reasoning_content` + `content` qua OpenAI-compatible API.

Production Tuning: Memory, Scheduling & Observability

- **`--mem-fraction-static`** (SGLang) / **`--gpu-memory-utilization`** (vLLM): chừa 5–8 GB baseline; quá thấp → OOM
- **`--chunked-prefill-size`**: giảm 2048–4096 khi prefill OOM (default 8192)
- **`--max-running-requests`** / `--max-num-seqs`: cap burst để tránh decode OOM
- **`--schedule-conservativeness`**: 0.3 aggressive · 1.0 default · 1.3 conservative
- Queue depth target: 100–2,000 (saturation >2K)

- **`--enable-metrics`** → Prometheus endpoint `:30000/metrics`
- Key metrics: `num_running_reqs`, `num_queue_reqs`, TTFT/TPOT histograms, cache-hit rate
- Scrape → Grafana dashboard; vLLM expose tương tự qua `prometheus_client`
- **`--log-requests`** (basic/full); **crash dump**: rolling 5-phút buffer
- Replay: `replay_request_dump.py` để reproduce lỗi

Tuning + Debug Flow — Start `conservativeness=1.0` → monitor queue depth → Grafana anomaly → `--log-requests` trace → crash dump replay → fix + redeploy.

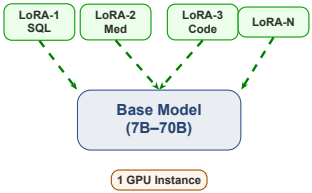
Disaggregated Prefill/Decode Serving



- **Mooncake** (Moonshot/Kimi): 100B+ tok/day, TTFT ↓75%
- **llm-d** v0.3: 2.2K tok/s/H200, K8s-native routing
- **NVIDIA Dynamo**: NIM-ready, disaggregated default engine
- **DistServe / Splitwise**: foundational papers (SOSP'24)

Lưu ý: KV transfer overhead ~10 GB/s. Lợi ích rõ khi workload prefill-heavy (long context, RAG). Không đáng cho short unique prompts.

Multi-LoRA Serving



- **Punica kernels**: merged Jan 2024 vào PyTorch
- **S-LoRA**: paged LoRA weights, swap per request
- **vLLM** `--enable-lora`: *N* adapters / 1 endpoint
- **SageMaker LMI-Dist**: managed multi-LoRA hosting
- **12**$\times$ throughput vs *N* separate single-model servers
- Overhead: $+2$ ms/token cho adapter application
- **SGLang**: Chunked SGMV (20–80% lat$\downarrow$); LoRA overlap loading (35% TTFT$\downarrow$)

Use case — 1 base model + nhiều domain adapters (SQL, y tế, code, finance) trên 1 GPU — tiết kiệm VRAM và serving cost so với *N* independent deployments.

Expert Parallelism: MoE Scaling

- Chia MoE expert weights qua nhiều GPUs (không replicate)
- Forward pipeline: dispatch → pre-permute → core runner → combine
- A2A backends: `--moe-a2a-backend deepep` (NVLink/IB), mooncake, nixl, flashinfer
- MoE runner: `--moe-runner-backend deep_gemm` hoặc cutlass
- Constraint: hầu hết backends yêu cầu `ep_size = tp_size`

- **Two-Batch Overlap (TBO)**: `--enable-two-batch-overlap` — 2 micro-batches xen kẽ → **2× throughput**
- **EPLB**: `--enable-eplb` — load balancer giảm GPU utilization variance
- **DeepEP mode**: `--deepep-mode auto` / `normal` / `low_latency`
- Ví dụ DeepSeek-V3 671B: `--tp 8 --ep 8 --enable-two-batch-overlap`

Two-Batch Overlap (TBO) — TBO xen kẽ A2A communication và GEMM computation trên 2 micro-batches song song — loại bỏ GPU idle time → **2× throughput** trên DeepSeek-V3.

Data Parallelism: DP, DPA & SMG Router

- **DP**: replicate toàn bộ model + KV cache → memory duplication
- **DPA**: chỉ replicate attention; MoE/FC layers chia sẻ qua EP → không duplicate KV cache
- **MLA benefit**: DPA + MLA = batch size lớn hơn, VRAM tiết kiệm đáng kể (DeepSeek V3/R1)
- Flags: `--dp-size N --enable-dp-attention`

- Gửi request đến instance có KV prefix cache phù hợp nhất
- Benchmark 8× A100 80 GB: throughput **+92%**, cache hit **+275%** (20% → 75%)
- Flags: `--router-policy cache_aware --cache-threshold 0.5`
- SMG: Rust-based, production-grade (dùng thay native DP router)

DeepSeek-V3 **DP+EP** **Config** — `--tp 8 --dp-size 8 --ep 8 --enable-dp-attention` kết hợp DPA + Expert Parallelism + cache-aware routing → **+92%** throughput.

Distributed Inference: Parallelism Strategy Guide

Strategy	Split	Best for	Tradeoff
Data Parallelism (DP)	Requests	Multi-user, replicated model	No KV cache sharing
Tensor Parallelism (TP)	Weights/layer	Large model, single-node	All-reduce sync mỗi layer
Pipeline Parallelism (PP)	Layers	Multi-node, 128K+ context	Bubble latency, micro-batch
Expert Parallelism (EP)	MoE experts	Mixtral / DeepSeek-V3 671B	Expert routing overhead
Disaggregated P/D	Prefill/Decode	Long RAG, prefill-heavy	KV transfer bandwidth cost

- `ray start --head` / `ray start --address=...` mỗi node
- `--tensor-parallel-size 4 --pipeline-parallel-size 2` → 8 GPU / 2 nodes
- NCCL collective ops; Ray tự quản lý device mesh
- `NCCL_IB_HCA=mlx5` cho InfiniBand NIC

- **TP within node**: NVLink 900 GB/s — all-reduce không bottleneck
- **PP across nodes**: P2P activation chịu được IB latency
- **Không TP qua nodes** trừ NVLink fabric (NVL72/GB200)
- **EP**: mỗi GPU giữ subset experts, A2A routing on-demand

Rule of Thumb — TP $\leq$ GPUs/node; PP = nodes; EP cho MoE. Ví dụ: `--tp 4 --pp 2 --ep 8` → cluster 2 nodes $\times$ 4 GPU phục vụ DeepSeek-V3 671B full precision.

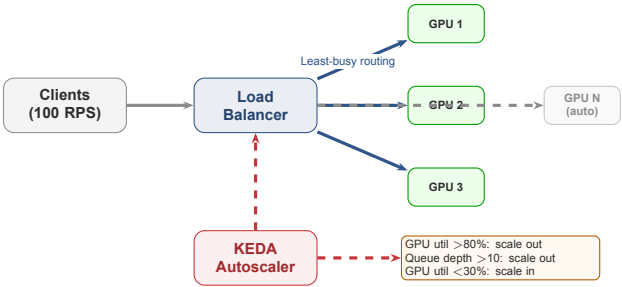
Pipeline Parallelism: Ultra-Long Context

- Chia model layers qua nhiều pipeline stages với P2P communication
- Chunked prefill: các nodes xử lý token chunks đồng thời $\rightarrow$ giảm TTFT long context
- `--pp-size N`: số stages; `--nnodes M`: số nodes
- `--enable-dynamic-chunking`: tự điều chỉnh theo prefix length
- Smooth factor env var (default 0.75, range 0.6–0.85)

- **DeepSeek-V3.1**: 4K fixed hoặc 12K dynamic (smooth=0.65)
- **Qwen3-235B**: 6K fixed hoặc 18K dynamic (smooth=0.8)
- Dynamic: dùng initial chunk 2–3$\times$ baseline để amortize overhead
- PCG tự động tắt khi bật PP
- Use case: 128K+ context trên multi-node GPU cluster

PP vs TP — PP $\neq$ TP: dùng PP khi model quá lớn cho single-node TP (DeepSeek-V3.1 full precision) hoặc cần 128K+ context — tận dụng multi-node inter-connect bandwidth.

Auto-scaling Architecture

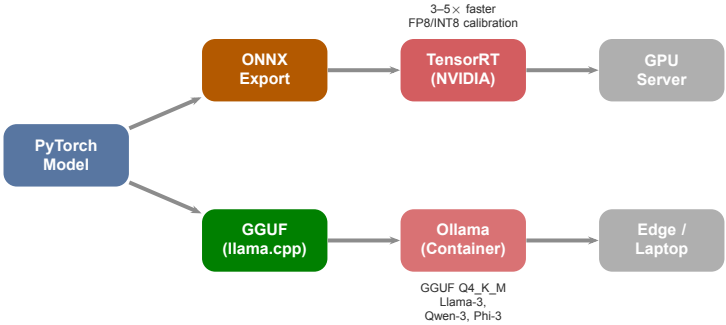


Scaling Strategies

- GPU utilization >80%: scale out
- Queue depth >10 requests
- KEDA + Knative: Event-Driven Autoscaling
- Scale-to-zero: 0 replicas khi no traffic — tiết kiệm chi phí đáng kể

- Least-busy routing: +30% vs round-robin
- Request batching: 50 ms window, +40% throughput
- Warm pool: N idle instances cho spikes
- Trade-off: cost vs cold start latency

Edge Deployment Flow



GGUF Levels: Q2_K (extreme, quality drop) · **Q4_K_M (recommended)** · Q6_K (near-lossless) · Q8_0 (max quality)

Models: Llama-3-8B, Qwen-3-8B, Phi-3-mini | **Ollama:** `ollama run llama3` — 1 lệnh duy nhất

Hardware Landscape 2026: Beyond H100

Chip	Peak tok/s (70B)	Memory	Niche / 2026 Status
NVIDIA H100 SXM	∼2,000	80 GB HBM3	Cloud standard; widely available
NVIDIA H200 SXM	∼3,000	141 GB HBM3e	+75% BW vs H100; in production
NVIDIA B200	∼4,500+	192 GB HBM3e	FP4 native; **sold out Q2 2026**
AMD MI300X/355X	∼2,200+	192–288 GB	Azure/AWS; MXFP4 (MI355X late 2025)
Groq LPU	Sub-100 ms TTFT	230 MB SRAM	Ultra-low latency; Groq Cloud
AWS Trainium2	∼1,500	96 GB HBM	Anthropic Rainier ∼500K chips

Lưu ý: B200/GB200 NVL72 sold out đến Q3 2026. AMD MI355X ra muộn nhưng MXFP4 native. **Anthropic Project Rainier**: ∼500K Trainium2 chips — không dùng NVIDIA GPU.

SLA Dashboard: Key Metrics

VINUNIVERSITY

120 ms
P50 Latency
Target: < 200 ms

380 ms
P95 Latency
Target: < 500 ms

850 ms
P99 Latency
Target: < 1000 ms

1,800
tokens/s per GPU
Benchmark: k6/locust

99.9%
Uptime
= 8.7h downtime/year

Production SLA: Best Practices

- Multi-AZ deployment + health checks
- Timeout 10–60s cho LLM generation
- Circuit breaker: fallback khi overloaded
- Graceful degradation: trả cached/shorter response, route sang smaller model

- Cost per 1M tokens: liên tục optimize
- Spot instances cho batch inference
- Scale-to-zero khi no traffic (KEDA)
- Right-size GPU: đừng dùng A100 cho 7B

Lưu ý: Benchmark với locust/k6 **trước** khi production. Không đoán latency — đo thực tế.

`Day20-Track2-ModelServing-Lab/` — chạy được trên Windows / macOS / Linux, low-spec laptop OK

- **00-setup**: hardware detection + cross-platform install
- **01-quickstart**: `llama-cpp-python` baseline P50/P95/P99
- **02-server**: OpenAI-compat `llama-server` + Prometheus + locust load test
- **03-milestone-integration**: nối endpoint với N16–N19

- **llama.cpp tuning**: build flags (AVX2/AVX-512, NEON), thread sweep, ctx-len sweep, GPU offload (Metal/CUDA/Vulkan), quant tradeoff Q2_K → Q8_0
- **MLX (macOS)**: Apple Silicon native runtime, optional

Lưu ý: Low-spec laptop (8 GB RAM, no GPU)? Vẫn chạy được toàn bộ core tracks. Bonus track hoạt động trên *mọi* CPU — càng ”yếu” càng học được nhiều về tối ưu.

Milestone 1: AI Infrastructure Platform

Tích hợp N16–N19 thành coherent AI infrastructure platform demo

1. Cloud setup: IaC + K8s cluster
2. Data pipeline: ingestion + processing
3. Lakehouse: Delta Lake + Medallion
4. Vector store: semantic search API
5. Feature store: online/offline
6. Model serving: optimized endpoint
7. Benchmark report: latency + cost
8. Live demo: end-to-end flow

Lưu ý: Submit trên LMS trước hết ngày. Demo live cho instructor trong lab session.

Chương 4 Recap: Hạ Tầng AI



Common mistakes: Skip validation → data quality issues · No time travel → no rollback · Static batching → poor throughput

Tổng kết — Key Takeaways

Những ý chính cần nhớ trước khi sang bài tiếp theo

1. Quantization 2026: FP8/NVFP4 (Hopper+/Blackwell) cho production cloud, AWQ 4-bit cho general, GGUF Q4_K_M cho edge.

2. Serving: vLLM v1 (APC default) + SGLang (RadixAttention) = production core. Disaggregated scale: llm-d / Dynamo. Local: Ollama.

3. Goodput @SLO (không phải throughput @peak) quyết định production success — benchmark P50/P95/P99 trước khi deploy.

Chương 5: CI/CD for AI Systems

“Từ hạ tầng sang vận hành — deploy AI models an toàn, tự động, liên tục.”

- Submit Milestone 1 đúng deadline
- Đọc trước: MLOps Principles — Google Cloud
- Cài sẵn GitHub Actions runner

Hỏi & Đáp

Câu hỏi nào về Quantization, vLLM/SGLang, SLA, hay Milestone 1?

VINUNIVERSITY

Cảm ơn!

AICB-P2T2 · Ngày 20

Model Serving & Inference Optimization

`lms.vinuni.edu.vn` · Slide & template trên LMS